



GEC-6818 实验箱

实验开发指导书

版权声明

本文档归属广州粤嵌通信科技股份有限公司（以下简称“粤嵌”）所有，并保留一切权利，非经粤嵌同意（书面形式），任何单位及个人不得擅自摘录本文档部分或者全部内容，违者将追究其法律责任。

作者	日期	版本	更新内容
粤嵌	2017.02.17	V1.0	

目 录

版权声明.....	2
目 录.....	1
第 1 章 编译环境搭建.....	1
1.1 选择系统平台.....	1
1.2 解压 android 源码包.....	1
1.3 安装交叉编译工具链.....	1
1.4 安装编译 android 系统所需工具.....	1
第 2 章 编译 android 源码.....	3
2.1 注意事项.....	3
2.2 编译 uboot.....	3
2.2.1 编译说明.....	3
2.3 编译 android 内核.....	4
2.4 编译 android 文件系统.....	4
2.4.1 编译说明.....	4
2.5 查看编译帮助.....	4
2.6 常见编译错误.....	5
2.7 编译脚本分析.....	6
第 3 章 烧写 android 映像文件.....	11
3.1 说明.....	11
3.1.1 注意事项.....	11
3.1.2 开发板启动顺序.....	11
3.2 Linux 下使用 fastboot 烧写.....	11
3.2.1 安装串口终端 minicom.....	11
3.2.2 安装 fastboot 工具.....	13
3.2.3 烧写 android 映像.....	13
3.3 Windows 下使用 fastboot 烧写.....	14
3.3.1 安装串口工具 secureCRT.....	14
3.3.2 安装 fastboot.....	17
3.3.3 烧写 android 映像.....	17
3.4 使用 SD 卡烧写.....	19
3.4.1 说明.....	19
3.4.2 Linux 下制作 SD 启动卡.....	19
3.4.3 Windows 下制作 SD 启动卡.....	21
3.4.4 使用 SD 启动卡烧写映像.....	27

第 1 章 编译环境搭建

1.1 选择系统平台

以下编译环境搭建基于 ubuntu14.04 64 位系统，建议使用相同的编译环境。

1.2 解压 android 源码包

从网盘中拷贝 android 源码包，放在自己的用户名目录，网盘中存放着 android5.1 的源码包，其名称为 6818android.tar.bz2。注意最好不要放在文件系统的根目录，这样会出现管理权限问题。

示例方法：在用户权限下执行如下命令：

```
tar xvf 6818android.tar.bz2
```

这时，整个 android 文件系统全部都放在了当前解压的目录中。至此，android 源码包解压完成。说明：源码包名称可能会因发布日期等有所不同，具体以网盘中实际名称为准。

1.3 安装交叉编译工具链

我们已经将交叉编译工具链集成到源码包中，无需再手动安装。交叉编译工具链在解压目录中的位置：

```
prebuilts/gcc/linux-x86/arm/arm-eabi-4.8
```

1.4 安装编译 android 系统所需工具

说明：如只需要编译 uboot、kernel，这步可以省略。若需要编译 android 系统以及 userdata，则必须要安装此工具。

使用如下命令安装编译所需的软件包：

```
sudo apt-get update
```

```
sudo apt-get install git-core gnupg flex bison gperf libssl1.2-dev libwxgtk3.0-dev
```

```
build-essential zip curl zlib1g-dev gcc-multilib g++-multilib genromfs  
libc6-dev-i386 libncurses5-dev x11proto-core-dev libx11-dev ccache libgl1-mesa-dev  
libxml2-utils xsltproc unzip gperf lsb-core lib32z1-dev lib32ncurses5-dev
```

使用如下命令安装 JDK1.7:

```
sudo add-apt-repository ppa:webupd8team/java  
sudo apt-get update  
sudo apt-get install oracle-java7-set-default
```

执行 `java -version` 指令查看当前 java 安装版本:

```
java -version  
java version "1.7.0_80"  
Java(TM) SE Runtime Environment (build 1.7.0_80-b15)  
Java HotSpot(TM) 64-Bit Server VM (build 24.80-b11, mixed mode)
```

到此, 编译 android 系统所需工具全部成功安装。

第 2 章 编译 android 源码

2.1 注意事项

注意：编译映像时一定要使用普通权限编译。

编译完成后映像文件会释放到 `out/release` 目录。编译完成后，最终我们需要的映像文件为：

`ubootpak.bin`, `boot.img`, `system.img`, `cache.img`, `userdata.img`。

文件作用说明：

- `ubootpak.bin`: bootloader, 用于引导内核, 它包含了 `2ndboot.bin`, `nsih` 以及 `u-boot.bin` 三个文件, 将他们打包成一个文件, 方便我们调试更新映像。
- `boot.img`: 它包含了内核 `zImage` 以及 `ramdisk.img` 两个文件。更新 `boot.img`, 等同于同时更新了 `uImage` 和 `ramdisk`。
- `system.img`: android 文件系统

2.2 编译 uboot

在 android 源码目录下执行如下命令编译 `uboot`, 编译完成后映像文件 `ubootpak.bin` 会释放到 `out/release` 目录。

```
/mk -u
```

2.2.1 编译说明

1、默认烧写 `uboot` 时, 需要烧写 `2ndboot.bin`, `nsih.txt` 以及 `u-boot.bin` 三个文件, 这里我们将三个文件打包成一个文件了, 并命名为 `ubootpak.bin`。pak 即 `package` 的意思。因此, 在平时调试时, 我们不再需要关注 `2ndboot.bin`, `nsih.txt` 以及 `u-boot.bin` 这三个文件。

2、mk 是我们写好的 `shell` 编译脚本, 在下文会对该编译脚本进行解析。

2.3 编译 android 内核

在 android 源码目录下执行如下命令编译 android 内核，编译完成后映像文件 boot.img 会释放到 out/release 目录。

```
/mk -k
```

说明：mk 是我们写好的 shell 编译脚本，在下文会对该编译脚本进行解析。

2.4 编译 android 文件系统

在 android 源码目录下执行如下命令编译 android 映像文件，编译完成后映像文件会释放到 out/release 目录。

```
/mk -s
```

提示：使用多线程编译提高速度：/mk -s -j=2，2 代表两个线程

2.4.1 编译说明

- 1、在编译文件系统前，必须先编译一次 uboot 与内核。
- 2、在修改文件系统中的部分文件后，由于编译器没有识别到文件更改，存在编译错误的情况，因此在编译文件系统前先编译一次内核。
- 3、编译完在在 out/target/product/x6818 的临时存放目录下会生成临时文件，在 out/release 目录下会生成如下文件：
ubootpak.bin, boot.img, system.img, cache.img, userdata.img, recovery.img。
- 4、mk 是我们写好的 shell 编译脚本，在下文会对该编译脚本进行解析。

2.5 查看编译帮助

执行如下指令可查询 mk 脚本使用方法：

```
/mk -h
```

2.6 常见编译错误

- 1、jdk 的版本不是 1.7,会出现如下编译错误:

```
Checking build tools versions...
/bin/bash: java: command not found
/bin/bash: javac: command not found
*****

You are attempting to build with the incorrect version
of java.

Your version is: .
The required version is: "1.7.x"

Please follow the machine setup instructions at
https://source.android.com/source/initializing.html
*****

build/core/main.mk:167: *** stop. Stop.
#### make failed to build some targets (3 seconds) ####
```

- 2、如果没有安装 gperf, 会出现如下编译错误:

```
/bin/bash: gperf: command not found
external/chromium_org/third_party/WebKit/Source/platform/make_platform_generated.target.
linu x-arm.mk:48: recipe for
target'out/target/product/x6818/obj/GYP/shared_intermediates/blink/platform/ColorData.cpp'
failed
make:
***[out/target/product/x6818/obj/GYP/shared_intermediates/blink/platform/ColorData.cpp]
Error 127
```

- 3、如果没有安装 libxml2-utils, 会出现如下编译错误:

```
/bin/bash: xmllint: command not found
build/core/Makefile:34: recipe for target
'out/target/product/x6818/system/etc/media_codecs_google_audio.xml' failed
make: *** [out/target/product/x6818/system/etc/media_codecs_google_audio.xml] Error 127
make: *** Waiting for unfinished jobs....
/bin/bash: xmllint: command not found
build/core/Makefile:34: recipe for target
'out/target/product/x6818/system/etc/media_codecs_google_video.xml' failed
make: *** [out/target/product/x6818/system/etc/media_codecs_google_video.xml] Error 127
Picked up JAVA_TOOL_OPTIONS: -javaagent:/usr/share/java/jayatanaag.jar
```


4、执行 make update-api 指令解决:

```
*****  
You have tried to change theAPI from what has been previously approved.  
To make these errors go away, you have two choices:  
1)You can add "@hide" javadoc comments to the methods, etc. listed in the  
errors above.  
2)You can update current.txt by executing the following command:  
make update-api  
To submit the revised current.txt to the main Android repository,  
you will need approval.  
*****
```

2.7 编译脚本分析

编译脚本 mk 内容及注释如下:

```
#!/bin/bash  
# 声明 java PATH 环境变量  
export PATH=/usr/lib/jvm/java-7-oracle/bin:$PATH  
  
# 指定常用路径  
BS_DIR_TOP=$(cd `dirname $0` ; pwd)  
BS_DIR_RELEASE=${BS_DIR_TOP}/out/release  
BS_DIR_TARGET=${BS_DIR_TOP}/out/target/product/x6818/  
BS_DIR_UBOOT=${BS_DIR_TOP}/uboot  
BS_DIR_KERNEL=${BS_DIR_TOP}/kernel/  
  
# 指定 uboot 和内核的交叉编译工具路径  
BS_CROSS_TOOLCHAIN_BOOTLOADER=${BS_DIR_TOP}/prebuilts/gcc/linux-x86/arm  
/arm-eabi-4.8/bin/arm-eabi-  
BS_CROSS_TOOLCHAIN_KERNEL=${BS_DIR_TOP}/prebuilts/gcc/linux-x86/arm/arm-e  
abi-4.8/bin/arm-eabi-  
  
# 指定 uboot, 内核, 文件系统以及 buildroot 的配置  
BS_CONFIG_BOOTLOADER_UBOOT=x6818_config  
BS_CONFIG_KERNEL=x6818_defconfig  
BS_CONFIG_FILESYSTEM=PRODUCT-x6818-userdebug  
  
# 编译前, 预先设置编译环境  
setup_environment()  
{  
LANG=C  
cd ${BS_DIR_TOP};
```

```
mkdir -p out/host/linux-x86/bin || return 1;
mkdir -p ${BS_DIR_TARGET}/boot;
#boot.tar.gz 和 generate_boot.tar.gz 保证没有编译 android 系统时，可以正常编译内核，
#生成 boot.img
[ -f "${BS_DIR_TOP}/out/host/linux-x86/bin/mkuserimg.sh" ] || { echo "tar generate
boot.img tools"; tar xvf tools/generate_boot.tar.gz -C
${BS_DIR_TOP}/out/host/linux-x86/bin;}
[ -f "${BS_DIR_TARGET}/boot/root.img.gz" ] || { echo "tar boot.tar.gz"; tar xvf
tools/boot.tar.gz -C ${BS_DIR_TARGET};}

PATH=${BS_DIR_TOP}/out/host/linux-x86/bin:$PATH;
mkdir -p ${BS_DIR_RELEASE} || return 1

[ -f "${BS_DIR_RELEASE}/x6818-sdmmc.sh" ] || { echo "copy sdfuse script..."; cp
${BS_DIR_UBOOT}/x6818-sdmmc.sh ${BS_DIR_RELEASE}/x6818-sdmmc.sh;}
}

#编译 uboot
build_bootloader_uboot()
{
# Compiler uboot
cd ${BS_DIR_UBOOT} || return 1
make distclean CROSS_COMPILE=${BS_CROSS_TOOLCHAIN_BOOTLOADER} ||
return 1
make ${BS_CONFIG_BOOTLOADER_UBOOT}
CROSS_COMPILE=${BS_CROSS_TOOLCHAIN_BOOTLOADER} || return 1
cp arch/arm/cpu/slsiap/devices/x6818-pmic.mk arch/arm/cpu/slsiap/devices/axp228_mfd.o
cp board/s5p6818/x6818/x6818-lcd.mk board/s5p6818/x6818/fastboot_lcd.o
cp net/x6818-eth.mk net/eth.o
make -j${threads} CROSS_COMPILE=${BS_CROSS_TOOLCHAIN_BOOTLOADER} ||
return 1

# Copy bootloader to release directory
cp -v ${BS_DIR_UBOOT}/ubootpak.bin ${BS_DIR_RELEASE}
cp -v ${BS_DIR_UBOOT}/readme.txt ${BS_DIR_RELEASE}
cp -v ${BS_DIR_UBOOT}/env.txt ${BS_DIR_RELEASE}

echo "^_^ uboot path: ${BS_DIR_RELEASE}/ubootpak.bin"
return 0
}

#编译内核映像
build_kernel()
{
```

```
export PATH=${BS_DIR_UBOOT}/tools:$PATH
# Compiler kernel
cd ${BS_DIR_KERNEL} || return 1
make ${BS_CONFIG_KERNEL} ARCH=arm
CROSS_COMPILE=${BS_CROSS_TOOLCHAIN_KERNEL} || return 1
make -j${threads} ARCH=arm
CROSS_COMPILE=${BS_CROSS_TOOLCHAIN_KERNEL} || return 1
make -j${threads} ARCH=arm
CROSS_COMPILE=${BS_CROSS_TOOLCHAIN_KERNEL} uImage || return 1
# Copy uImage to release directory
cp -v ${BS_DIR_KERNEL}/arch/arm/boot/uImage ${BS_DIR_RELEASE}
echo "^_ kernel path: ${BS_DIR_RELEASE}/uImage"
# generate boot.img
cd ${BS_DIR_TOP} || return 1
echo 'boot.img ->' ${BS_DIR_RELEASE}
# Make boot.img with ext4 format, 64MB
cp -v ${BS_DIR_RELEASE}/uImage ${BS_DIR_TARGET}/boot
mkuserimg.sh -s ${BS_DIR_TARGET}/boot ${BS_DIR_TARGET}/boot.img ext4 boot
67108864
cp -av ${BS_DIR_TARGET}/boot.img ${BS_DIR_RELEASE} || return 1;
return 0
}

#编译 android 文件系统
build_system()
{
cd ${BS_DIR_TOP} || return 1
source build/envsetup.sh || return 1
make -j${threads} ${BS_CONFIG_FILESYSTEM} || return 1
# Make boot.img
# Create boot directory
mkdir -p ${BS_DIR_TARGET}/boot || return 1
# Copy some images to boot directory
if [ -f ${BS_DIR_RELEASE}/uImage ]; then
    cp -v ${BS_DIR_RELEASE}/uImage ${BS_DIR_TARGET}/boot
fi
if [ -f ${BS_DIR_TARGET}/ramdisk.img ]; then
    cp -v ${BS_DIR_TARGET}/ramdisk.img ${BS_DIR_TARGET}/boot/root.img.gz
fi
if [ -f ${BS_DIR_TARGET}/ramdisk-recovery.img ]; then
    cp -v ${BS_DIR_TARGET}/ramdisk-recovery.img ${BS_DIR_TARGET}/boot
fi
# Make boot.img with ext4 format, 64MB
mkuserimg.sh -s ${BS_DIR_TARGET}/boot ${BS_DIR_TARGET}/boot.img ext4 boot
```

67108864

```
# Copy to release directory
cp -av ${BS_DIR_TARGET}/boot.img ${BS_DIR_RELEASE} || return 1;
cp -av ${BS_DIR_TARGET}/system.img ${BS_DIR_RELEASE} || return 1;
cp -av ${BS_DIR_TARGET}/cache.img ${BS_DIR_RELEASE} || return 1;
cp -av ${BS_DIR_TARGET}/recovery.img ${BS_DIR_RELEASE} || return 1;
cp -av ${BS_DIR_TARGET}/userdata.img ${BS_DIR_RELEASE} || return 1;

return 0
}

#默认使用单核编译系统，在编译时可以使用./mk -j=n 提高效率，n 为核数量
threads=1
uboot=no
kernel=no
system=no

if [ -z $1 ]; then
    uboot=yes
    kernel=yes
    system=yes
    buildroot=yes
fi

#根据编译脚本传参选择要编译的映像
while [ "$1" ]; do
    case "$1" in
        -j=*)
            x=$1
            threads=${x#-j=}
            ;;
        -u|--uboot)
            uboot=yes
            ;;
        -k|--kernel)
            kernel=yes
            ;;
        -s|--system)
            system=yes
            ;;
        -a|--all)
            uboot=yes
            kernel=yes
    esac
    shift
```

```
system=yes
;;
-h|--help)
    cat >&2 <<EOF
Usage: ./mk [OPTION]
Build script for compile the source of telechips project.

-j=n                using n threads when building source project (example: -j=16)
-u, --uboot         build bootloader uboot from source
-k, --kernel        build kernel from source
-s, --system        build android file system from source
-a, --all           build all, include anything
-h, --help          display this help and exit
EOF
    exit 0
    ;;
*)
    echo "./mk: Unrecognised option $1" >&2
    exit 1
    ;;
esac
shift
done

setup_environment || exit 1

#执行编译 uboot 映像
if [ "${uboot}" = yes ]; then
    build_bootloader_uboot || exit 1
fi

#执行编译内核映像
if [ "${kernel}" = yes ]; then
    build_kernel || exit 1
fi

#执行编译 android 文件系统映像
if [ "${system}" = yes ]; then
    build_system || exit 1
fi

exit 0
```

第 3 章 烧写 android 映像文件

3.1 说明

3.1.1 注意事项

使用 fastboot 烧写 android 映像时，板子上必须存在 uboot，因为烧写时需要使用 uboot 上的 fastboot 功能，在板子不存在 uboot 时，请使用 SD 卡烧写方式。

使用 fastboot 烧写时，电脑上必须存在串口接口或者拥有 usb 转串口模块，使其连接电脑与开发板，让电脑能够通过串口与开发板通信

3.1.2 开发板启动顺序

6818 开发板与实验箱硬件配置固定了开发板启动顺序如下：

- 1st: 从 TF 卡启动
- 2nd: 从 EMMC 启动
- 3rd: 从 USB 启动

开发板上电后首先从 TF 卡启动，若 SD0 插入了启动卡则从 SD 启动；如果 SD0 未插卡或者插入的不是启动卡，则启动失败；然后从板载 EMMC(SD2) 启动，若 EMMC 中已经烧录固件则启动成功，否则启动失败，最后尝试从 USB 启动。

3.2 Linux 下使用 fastboot 烧写

3.2.1 安装串口终端 minicom

1、使用如下指令安装：

```
sudo apt-get install minicom
```

2、如果是使用 USB 转串口模块，目前市面上大多都是 pl2303 方案，需要输入如下命令查询驱动是否正常加载：

```
lsmod |grep pl2303
```

返回如下信息则加载正常：

```
lqm@lqm:~$ lsmod |grep pl2303
pl2303 11756 1
usbserial 33100 3 pl2303
```

3、查看串口设备名：

```
dmesg | tail -f
```

返回：

```
ERROR! H2M_MAILBOX still hold by MCU. command fail
---> RTMPFreeTxRxRingMemory
<--- RTMPFreeTxRxRingMemory
RTUSB disconnect successfully
usb 2-4: USB disconnect, address 3
pl2303 ttyUSB0: pl2303 converter now disconnected from ttyUSB0
pl2303 2-4:1.0: device disconnected
usb 2-4: new full speed USB device using ohci_hcd and address 5
pl2303 2-4:1.0: pl2303 converter detected
usb 2-4: pl2303 converter now attached to ttyUSB0
exit 0
```

其中 ttyUSB0 就是串口设备名。

4、输入命令配置串口参数：

```
sudo minicom -s
```

选择 Serial port setup,

- 设置串口终端：选择 A，输入正确的串口终端，如果直接使用串口，通常设置为 ttyS0，如果使用 USB 转串口，通常设置为 ttyUSB0，回车。ttyS0 以及 ttyUSB0 中的 0 代表串口 0。
- 设置波特率以及数据位：选择 E，输入 115200 8N1，回车。115200 代表波特率，8N1 代表 8 个数据位，1 个停止位。
- 设置流控：选择 F 和 G，都设置为 No，表示不使用流控，回车。

选择 Save setup as dfl 保存设置。

5、常见使用问题：

非正常关闭 minicom，会在/var/lock 下创建几个文件 LCK*，这几个文件阻止了 minicom 的运行，将它们删除后即可恢复正常。

3.2.2 安装 fastboot 工具

1、执行如下指令安装 fastboot:

```
sudo apt-get install android-tools-fastboot
```

2、在使用 fastboot 时，需要获取 root 权限，为了解除这个限制，可以使用如下方法，让使用 fastboot 时不需要管理员权限。

到/etc/udev/rules.d/目录下，新建 51-android.rules 文件，内容如下：

（注意，OWNER 里面填的” gec” 务必换成自己 ubuntu 系统的用户名）

```
# adb protocol on passion (Nexus One)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e12",
MODE="0666",OWNER="gec"
# adb protocol on crespo/crespo4g (Nexus S)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e22",
MODE="0666",OWNER="gec"
# fastboot protocol on crespo/crespo4g (Nexus S)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e20",
MODE="0666",OWNER="gec"
# fastboot protocol on stingray/wingray (Xoom)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="708c",
MODE="0666",OWNER="gec"
# fastboot protocol on maguro/toro (Galaxy Nexus)
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="4e30",
MODE="0666",OWNER="gec"
# fastboot protocol on x210/x4412/x6818
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", ATTR{idProduct}=="0002",
MODE="0666",OWNER="gec"
```

3.2.3 烧写 android 映像

1、使用串口线或 USB 转串口模块连接开发板与电脑。

2、执行如下指打开 minicom:

```
minicon
```

3、打开开发板电源，在 minicom 中查看串口打印的启动信息，在 uboot 启动的 3 秒内按任意键进入 uboot 命令行模式，执行如下指并回车：

fast

minicom 终端下将打印如下信息：

```
Fastboot Partitions:
mmc.2: ubootpak, img : 0x200, 0x78000
mmc.2: 2ndboot, img : 0x200, 0x4000
mmc.2: bootloader, img : 0x8000, 0x70000
mmc.2: boot, fs : 0x100000, 0x4000000
mmc.2: system, fs : 0x4100000, 0x2f200000
mmc.2: cache, fs : 0x33300000, 0x1ac00000
mmc.2: misc, fs : 0x4e000000, 0x800000
mmc.2: recovery, fs : 0x4e900000, 0x1600000
mmc.2: userdata, fs : 0x50000000, 0x0
Support fstype : 2nd boot factory raw fat ext4 emmc nand ubi ubifs
Reserved part : partmap mem env cmd
DONE: Logo bmp 311 by 300 (3bpp), len=280854
DRAW: 0x47000000 -> 0x46000000
Load USB Driver: android
Core usb device tie configuration done
OTG cable Connected!
-----
```

4、插入 micro USB 线连接到电脑。

5、另外开启一个 ubuntu 命令行终端，输入如下指令更新 android 系统各个部分需要的映像：

fastboot flash ubootpak ubootpak.bin	(uboot 部分)
fastboot flash boot boot.img	(内核部分)
fastboot flash system system.img	(文件系统部分)
fastboot flash cache cache.img	(系统缓存部分)
fastboot flash userdata userdata.img	(用户空间部分)

执行完以上指令，即可正常启动 android 系统了。每执行一条指令，在液晶屏上都会有相应的界面提示，用户可以很清晰的观察升级的状态。

3.3 Windows 下使用 fastboot 烧写

3.3.1 安装串口工具 secureCRT

1、下载并安装 secureCRT 工具，打开工具，点击左上角“快速链接”按钮：

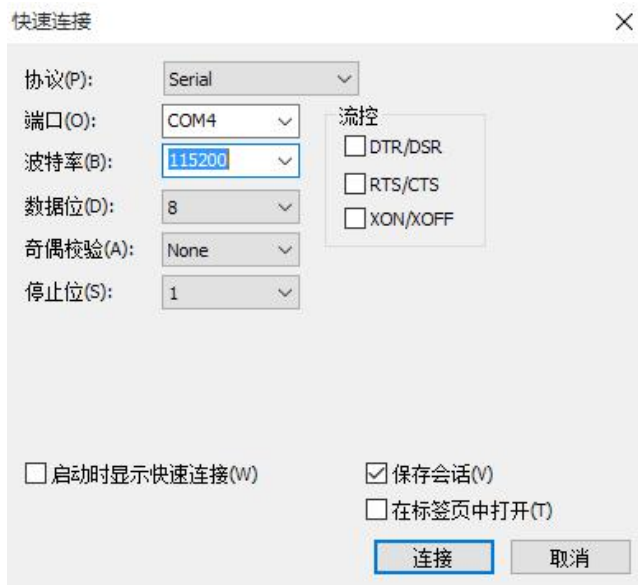


- 2、使用串口线或 USB 转串口模块连接开发板与电脑，打开 Windows 的设备管理器，查看串口端口号：

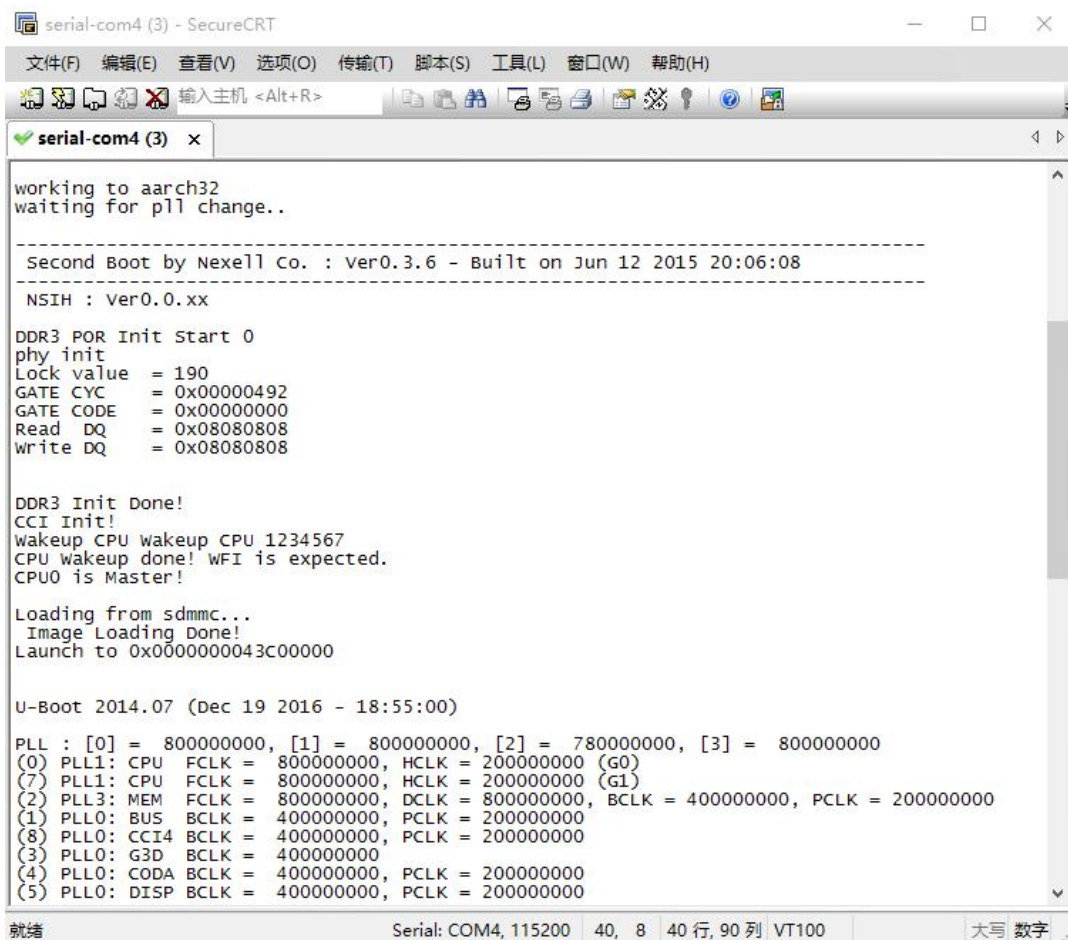


可以看到串口端口号为 COM4。

- 3、回到 secureCRT 工具界面，设置“快速链接”的配置。选择协议为 Serial，端口为 COM4，波特率为 115200，取消勾选流控 RTS/CTS：



- 4、点击连接后，打开开发板电源，secureCRT 终端输出开发板启动信息，说明 secureCRT 配置完成：



3.3.2 安装 fastboot

1、说明：在多数情况下，在 Windows 下使用 fastboot 工具烧写并不需要把 fastboot 安装到系统中，只需要解压 fastboot 工具并在解压目录中运行工具进行烧写即可。

2、如果需要在任意目录都可以使用 fastboot 工具，则可以使用如下方法安装：将资料盘 tools 目录中的 fastboot.rar 文件解压到 C:\Windows\System32 和 C:\Windows\SysWOW64（winXP 无此目录）目录即完成安装。

3.3.3 烧写 android 映像

- 1、使用串口线或 USB 转串口模块连接开发板与电脑。
- 2、打开 secureCRT 终端连接开发板串口。
- 3、打开开发板电源，在 secureCRT 中查看串口打印的启动信息，在 uboot 启动的 3 秒内按任意键进入 uboot 命令行模式，执行如下指并回车：

```
fast
```

secureCRT 终端下将打印如下信息：

```
Fastboot Partitions:
mmc.2: ubootpak, img : 0x200, 0x78000
mmc.2: 2ndboot, img : 0x200, 0x4000
mmc.2: bootloader, img : 0x8000, 0x70000
mmc.2: boot, fs : 0x100000, 0x4000000
mmc.2: system, fs : 0x4100000, 0x2f200000
mmc.2: cache, fs : 0x33300000, 0x1ac00000
mmc.2: misc, fs : 0x4e000000, 0x800000
mmc.2: recovery, fs : 0x4e900000, 0x1600000
mmc.2: userdata, fs : 0x50000000, 0x0
Support fstype : 2nd boot factory raw fat ext4 emmc nand ubi ubifs
Reserved part : partmap mem env cmd
DONE: Logo bmp 311 by 300 (3bpp), len=280854
DRAW: 0x47000000 -> 0x46000000
Load USB Driver: android
Core usb device tie configuration done
OTG cable Connected!
-----
```

- 4、插入 micro USB 线连接到电脑。

5、解压 fastboot 工具压缩包到一个目录下,把 android 映像文件 ubootpak.bin、boot.img、system.img、cache.img、userdata.img 全部复制到该目录中。

6、右键使用记事本编辑 Windows 脚本文件 auto.bat,查看烧写映像文件名是否与我们编译出来的 android 映像文件名相同,不相同则重命名 android 映像文件名。脚本文件 auto.bat 的内容:

```
fastboot flash ubootpak ubootpak.bin
fastboot flash boot boot.img
fastboot flash system system.img
fastboot flash cache cache.img
fastboot flash userdata userdata.img
```

7、确认无误后,退出编辑,右键使用管理员权限打开 auto.bat,可以看到 Windows 下会打开命令终端,打印出如下信息:

```
C:\Windows\system32\cmd.exe

F:\工作备份\6818\X6818_A\Image\x6818-android>fastboot flash ubootpak ubootpak.bin
sending 'ubootpak' (332 KB)... OKAY
writing 'ubootpak'... OKAY

F:\工作备份\6818\X6818_A\Image\x6818-android>fastboot flash boot boot.img
sending 'boot' (16556 KB)... OKAY
writing 'boot'... OKAY

F:\工作备份\6818\X6818_A\Image\x6818-android>fastboot flash system system.img
```

6、在 secureCRT 终端下,会打印出如下信息,说明烧写成功:

```
Core usb device tie configuration done
OTG cable Connected!
-----
Starting download of 340728 bytes
downloading 337920 -- 99% complete.
downloading of 340728 bytes to 0x48000000 (0xf8000000) finished
Flash : ubootpak
head boot dev = 2
update mmc.2 type 2ndboot = 0x200(0x1) ~ 0x532f8(0x29a): Done
Flash : ubootpak - DONE
Starting download of 16953576 bytes
downloading 16784896 -- 99% complete.
downloading of 16953576 bytes to 0x48000000 (0xf8000000) finished
Flash : boot
Flash : boot - DONE
Starting download of 443044932 bytes
downloading 438615040 -- 99% complete.
downloading of 443044932 bytes to 0x48000000 (0xf8000000) finished
Flash : system
Flash : system - DONE
Starting download of 8962352 bytes
downloading 8873472 -- 99% complete.
downloading of 8962352 bytes to 0x48000000 (0xf8000000) finished
Flash : cache
Flash : cache - DONE
Starting download of 97924564 bytes
downloading 96946176 -- 99% complete.
downloading of 97924564 bytes to 0x48000000 (0xf8000000) finished
Flash : userdata
Flash : userdata - DONE
```

7、烧写完成后,Windows 命令框会自动退出,开发板会重新启动。执行完以上指令,即可正常启动 android 系统了。每执行一条指令,在液晶屏上都会有相应的界面提示,用户可以很清晰的观察升级的状态。

3.4 使用 SD 卡烧写

3.4.1 说明

使用 SD 卡烧写必须准备一张不小于 2GB 的 micro SD 卡以及 micro SD 卡读卡器。使用 SD 卡烧写会清空 SD 卡中原有内容，请提前做好备份。

3.4.2 Linux 下制作 SD 启动卡

1、输入如下命令查看 linux 系统原有的设备节点。

```
cat /proc/partitions
```

返回：

major	minor	#blocks	name
8	0	36700160	sda
8	1	512000	sda1
8	2	36187136	sda2
253	0	34144256	dm-0
253	1	2031616	dm-1

2、使用读卡器连接 SD 卡与电脑，输入如下命令查看设备节点，经过对比原有设备节点获得 SD 卡设备节点名为 sdb

```
cat /proc/partitions
```

返回：

major	minor	#blocks	name
8	0	36700160	sda
8	1	512000	sda1
8	2	36187136	sda2
253	0	34144256	dm-0
253	1	2031616	dm-1
8	16	3879936	sdb
8	17	3875840	sdb1

3、输入如下命令，修改 SD 卡中的分区


```
sudo fdisk /dev/sdb
```

返回:

```
Command (m for help):
```

输入 **d** 并回车, 删除所有分区。返回:

```
Selected partition 1
```

```
Command (m for help):
```

输入 **w** 并回车, 保存所有已经修改的分区信息。返回:

```
The partition table has been altered!
```

```
Calling ioctl() to re-read partition table.
```

```
Syncing disks.
```

拔掉 SD 卡, 再插入 PC 机上, 输入如下命令查看现有的设备节点:

```
cat /proc/partitions
```

返回:

major	minor	#blocks	name
8	0	36700160	sda
8	1	512000	sda1
8	2	36187136	sda2
253	0	34144256	dm-0
253	1	2031616	dm-1
8	16	3879936	sdb

至此, SD 卡原分区/dev/sdb1 被删除。

5、使用 **gparted** 工具给 SD 卡预留 256M 空间, 用于存放 **uboot** 映像。使用如下命令安装 **gparted** 工具:

```
sudo apt-get install gparted
```

使用如下命令使用 **gparted** 打开 SD 卡分区表:

```
sudo gparted /dev/sdb
```

弹出:



6、选择分区->新建，预留 256M 空间给 uboot，剩下的分区使用 fat32 格式，设置如下图所示，设置完成后点击添加，选择菜单中的应用全部操作，完成 SD 卡的分区。



7、进入映像生成目录 out/release，执行如下指令烧写 uboot 到 SD 卡：（注意，这里/dev/sdb 为 SD 卡的节点，可查询节点名称后再执行下面的烧写脚本。）

```
sudo ./x6818-sdmmc.sh /dev/sdb ubootpak.bin
```

返回如图：

```
594+1 records in
595+0 records out
304640 bytes (305 kB) copied, 0.485139 s, 628 kB/s
^^ The image is fused successfully
```

这时，该 SD 卡就可以引导开发板启动 uboot 了。

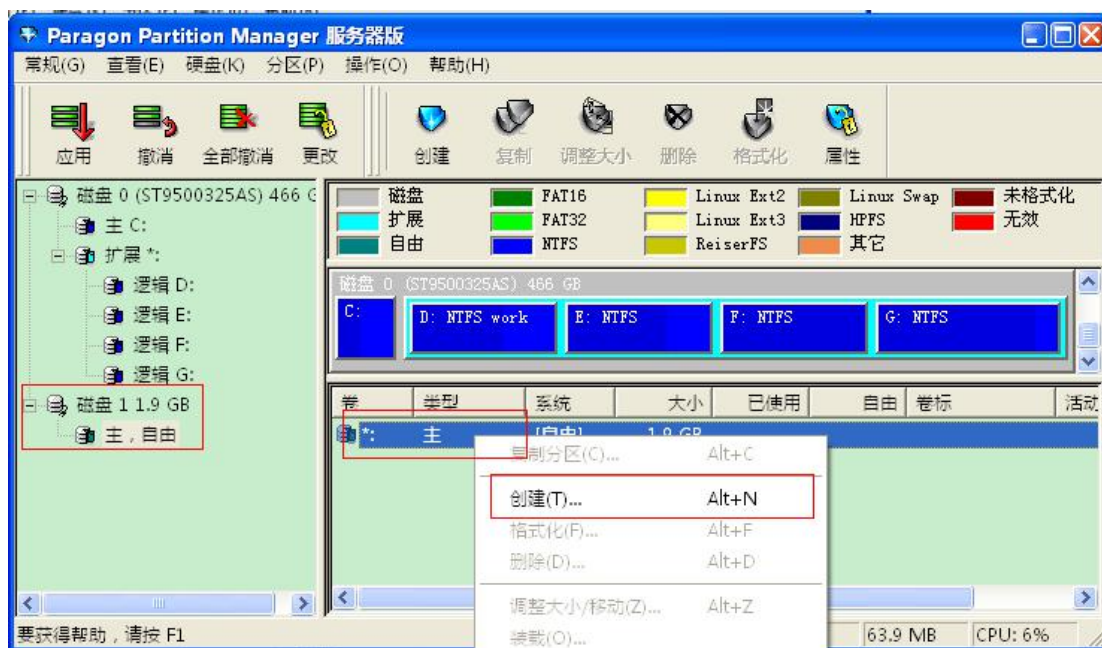
3.4.3 Windows 下制作 SD 启动卡

1、注意事项：WinPM 这个软件是为 windowsXP 而设置的，使用 windows7 或者 windows8 或者 windows10 的同学记得选择该软件，右键 ——> 兼容性建议 ——> 尝试建议性操作 ——> 启动程序）

2、整个流程是在 Windows 下要先通过 WinPM 这个软件来对 SD 卡进行分区，

然后再使用 IROM_Fusing_Tool_6818.exe 的软件，将我们编译好的 bootloader 固化到 SD 卡里面，然后再选择从 SD 卡启动我们的开发板进行烧写。以下是详细步骤：

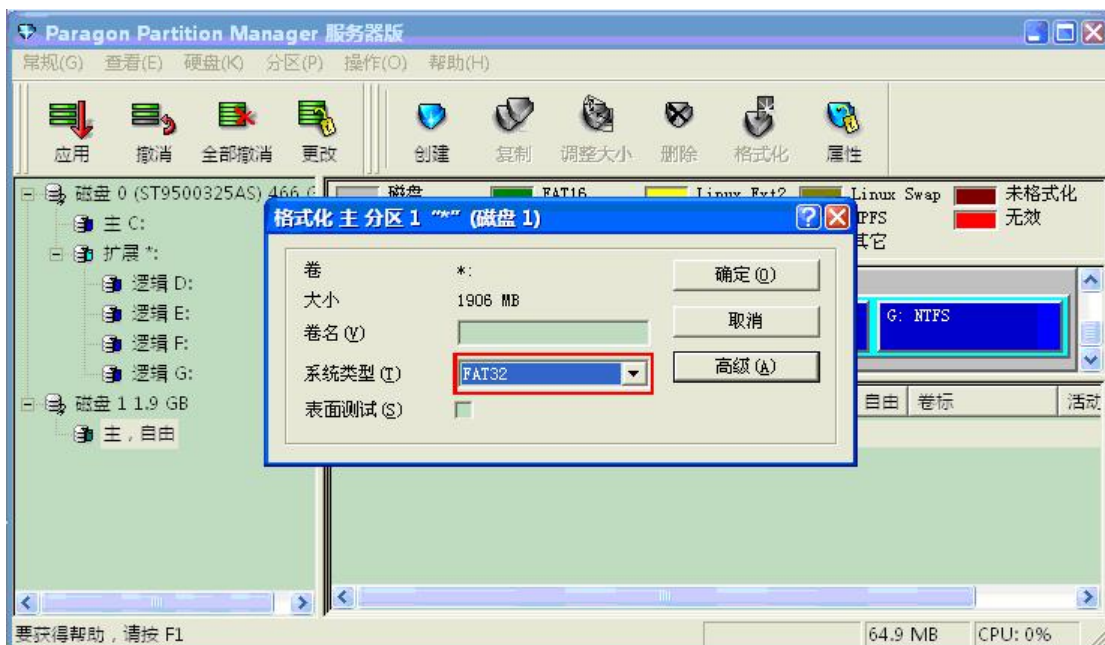
- 1) 打开 VinPM，这个软件位于光盘文件的 Tools 目录下面。
- 2) 选择 SD 卡，注意：选择你正确的 SD 卡盘符，在制作 SD 卡启动卡的时候会破坏 SD 里面的数据，请预先保存好你的数据。



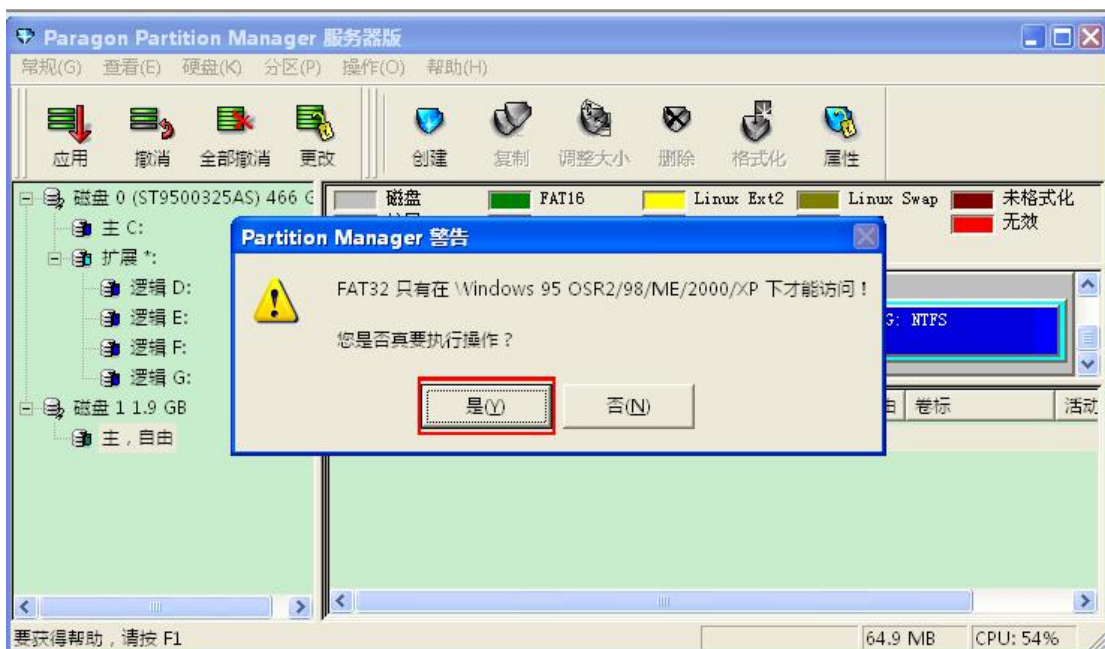
- 3) 预留 250M 的空间给 uboot。



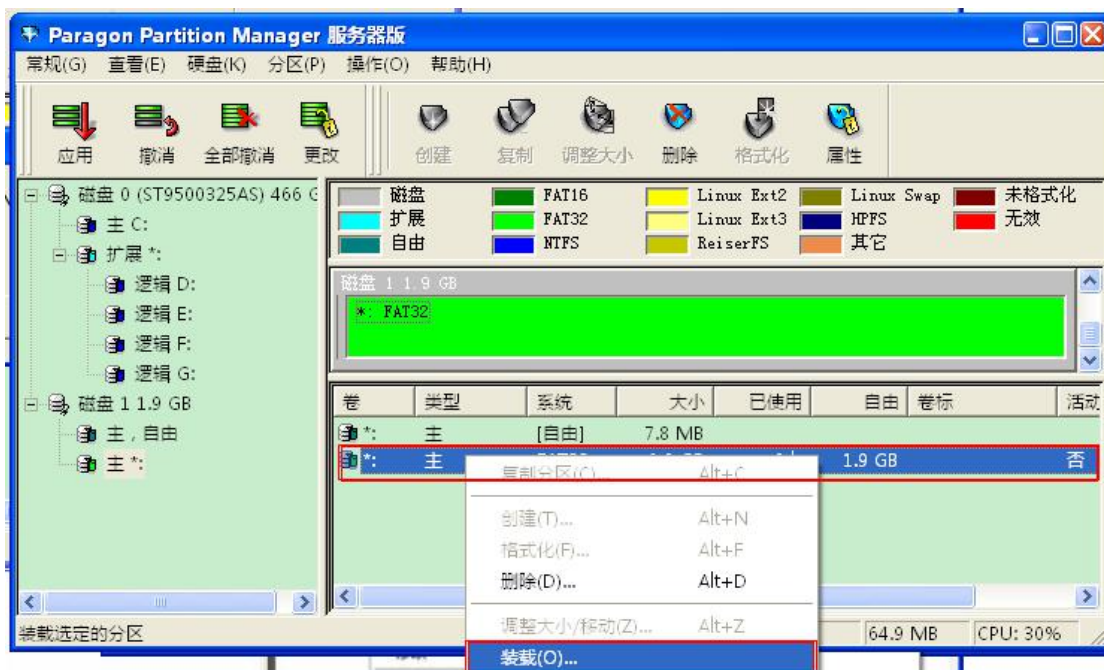
4) 选择是 FAT32 的格式



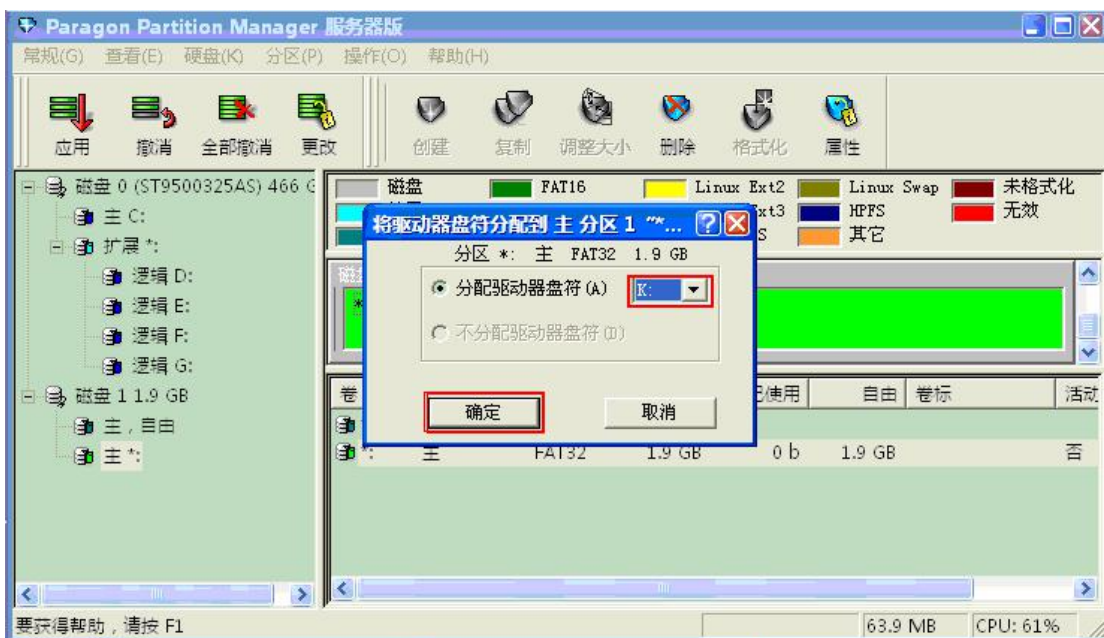
5) 选择确定



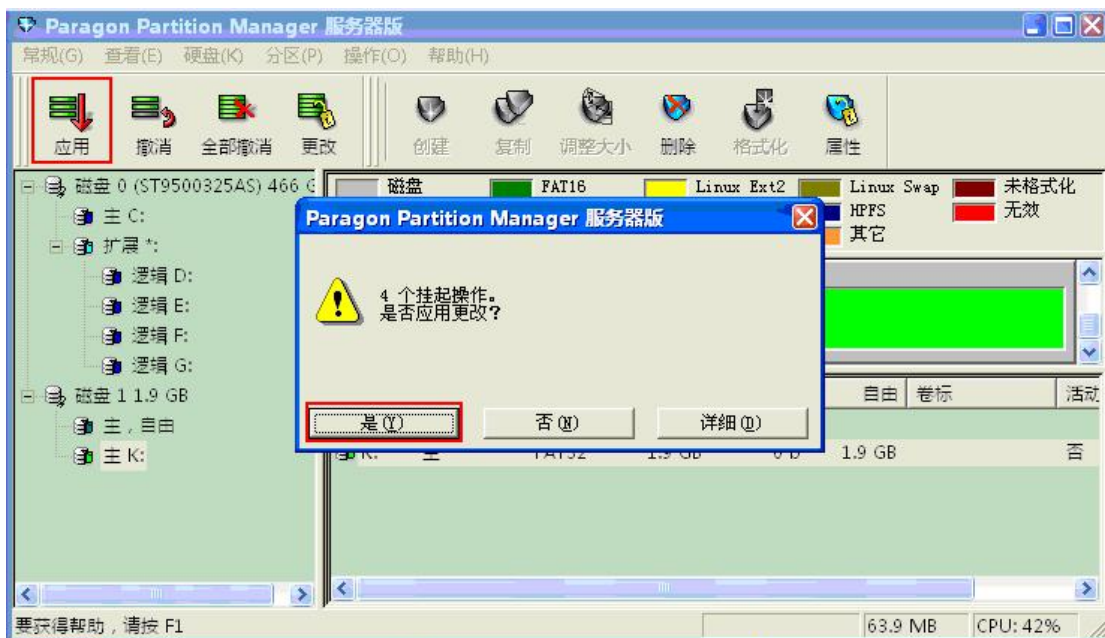
6) 再选择装载



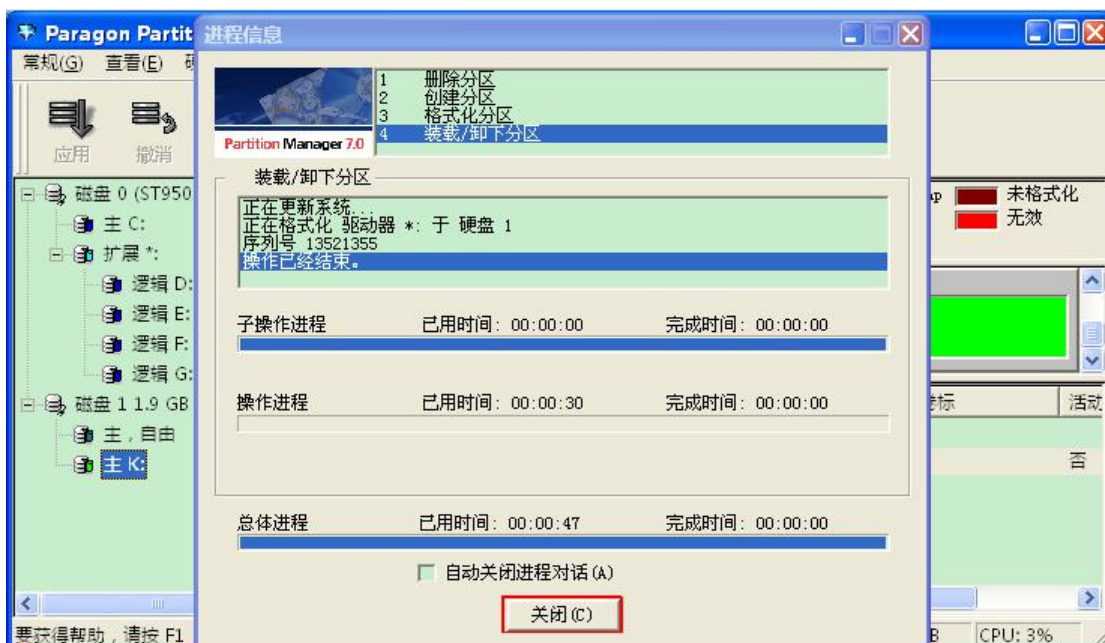
7) 系统会给我们新建的这个盘分一个盘符，我设置为 K，点击确定即可



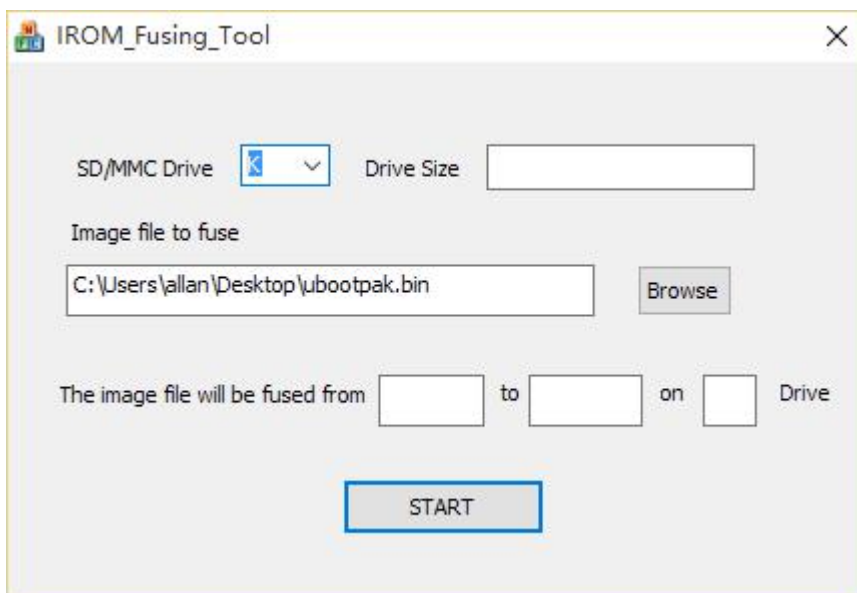
8) 对我们的 SD 卡分区好之后, 点击应用点击确定, 就可以对我们的 SD 卡进行分区, 建立分区表信息。



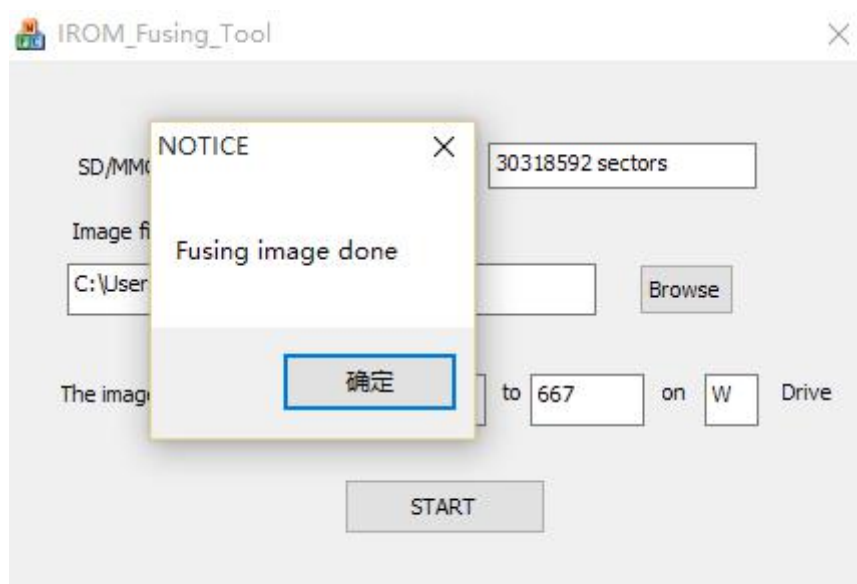
9) 分区完成之后, 点击关闭即可。



10) 对 SD 卡分好区之后，以管理员权限运行 IROM_Fusing_Tool_6818.exe 工具。点击 Browse 浏览到已经编译好的 uboot，所需烧写的 uboot 文件在 out/release 目录下，文件名字为：ubootpak.bin，如下图所示：



11) 选择我们需要烧写的选择正确的盘符（我们上面设置 SD 的盘符是 K，这里选择 K）。然后点击 START，显示 Fusing image done 表示烧写完成。完成 uboot



至此，SD 启动卡的制作已经完成。

3.4.4 使用 SD 启动卡烧写映像

1、使用读卡器连接已经制作好的 SD 启动卡与电脑，SD 启动卡的制作方法请参考上文《Linux 下制作 SD 启动卡》或《Windows 下制作 SD 启动卡》。

2、在 SD 卡中，新建名为 x6818-android 的文件夹，把编译的目标文件生成目录 out/release 中的 ubootpak.bin, boot.img, system.img, userdata.img 以及 cache.img, env.txt 六个文件复制并粘贴到该目录下，如图：

名称	修改日期	类型	大小
 boot.img	2017/2/16 10:47	光盘映像文件	16,557 KB
 cache.img	2017/2/13 12:27	光盘映像文件	8,753 KB
 env.txt	2017/2/15 18:18	文本文档	1 KB
 system.img	2017/2/16 10:48	光盘映像文件	428,650 KB
 ubootpak.bin	2017/2/16 10:49	BIN 文件	334 KB
 userdata.img	2017/2/16 10:49	光盘映像文件	95,630 KB

3、env.txt 为传给 uboot 的启动参数，他记录了烧写映像时需要烧写的文件文件名字，以及是否需要烧写。我们在使用时，可以编辑该文件，把需要更新的映像设置为 1，不需要更新的映像设置为 0 即可。env.txt 内容如下：

```
bootcmd=ext4load mmc 2:1 0x48000000 uImage;ext4load mmc 2:1 0x49000000
root.img.gz;bootm 0x48000000
bootargs=lcd=vs070cxn tp=gs1x680
ubootpak=1
boot=1
system=1
userdata=1
cache=1
```

4、将 SD 卡插到开发板的 SD0 卡槽，打开开发板电源，这时系统将会自动升级。等待数秒后，开发板会自动进行烧写。烧写完毕将会自动重启，进入 android 系统。

说明：默认 uboot 会自动判断 SD 卡的 x6818d -android 目录中的映像文件是否有更新，一旦发现映像和烧写到开发板上的不同，则会自动更新映像，否则不会再次更新；如果需要强制进入升级模式，开机上电时按住返回键（K1，调试串口会打印 force update）即可重新升级。